INF205 Task 3.6

Problem 2

2.1 Graph data structure based on incidence lists

Implemented a graph using incidence lists where nodes are objects with unique string labels and edges with non-unique. Each node has its own incidence list of edges connected to it. While also maintaining references to source and target nodes. Insert_edge(...) creates nodes.

2.2 File input/output

The graph files use triple notation. In this task read and write function was implemented (loadFromFile(..), saveToFile(...)). The files are space separated and use a period on each line.

2.3 Abstract class as an interface

Defined AbstractGraph class as pure virtual interface. The benefits of this is that algorithms work with the class pointer, while also improving reuse and maintainability

2.4 Matrix based data structure

Implemented an alternative implementation using adjacency matrix. The matrix implementation has much faster lookup of edges as it has a 3D vector. The vector of edges between nodes can easily be looked up in the "2D" matrix of node i and j (A[i][j]). While the lookup is quite effective the storage used is quite ineffective. The method should be used for dense graphs, low number of nodes and memory is not critical.'

2.5 delete edges and nodes

Disconnect(...) removes all edges from source node to target node, cleans up isolated nodes. This function works with both implementations.

Remove_node(...) deletes node from the graph, removes all incident edges and removes newly isolated nodes.

The isolated nodes are found by checking if they have no incoming and outgoing edges.

2.6 Rule of five

All container classes must have destructor, copy constructor, copy assignment operator, move constructor and move assignment operator. The implementation uses deep copies where necessary, this ensures that there are no shared pointers during the operations. The methods also use proper cleanup of allocated memory, ensuring no memory leakage.

Problem 3

3.1 Strongly connected components

The Tarjan algorithm aims to find all strongly connected components/nodes utilizing a depth-first search. It will identify all strongly connected components (SCC) in one traversal. SCC is defined as a maximal set of nodes where every node is reachable from every other node in the set. The algorithm returns vector<vector<Node*>> representing each SCC. The implementation of the algorithm works the same for LinkedGraph and MatrixGraph. The code can be run silent

using -s flag or verbose mode using -v, where it prints step by step. -- linked or –matrix tells the program which structure to use.

3.2. Query for diamond-shaped paths

This task aims to identify node pairs with two edge paths. One path will have even labels (r0, r2, ...) and the other will have odd labels (r1, r3, ...). The paths taken through other nodes using edges will create a diamond shape. The query therefore finds all the node pairs on the paths. The paths must also connect the source and target node. If not, the pair is skipped, and the node is unreachable.

The paths used by the algorithm need to be read from a query file. The query length can be selected by the user, to adjust for the scaling of data.

3.3 Demonstration using GNU make

For demonstration of the code three files have been made. As for problem 2 all is done through the graph-dem-pro2 and can be called as «./graph-demo-pro2 <pro21|pro22|pro23|pro24|pro25|pro26>» to test all the tasks at once.

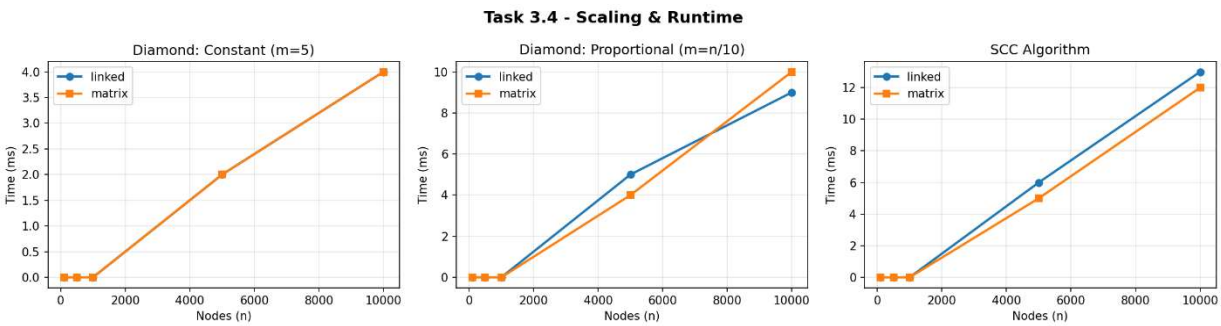
For 3.1 «./graph-benchmark-scc scc-test-data/scc02.dat --silent --linked» can be called to run the demo using the Tarjan algorithm. And «./graph-benchmark-diamond knowledge-test-data/kb03a.dat knowledge-test-data/qy03a.dat --silent --linked» running a demo for 3.2 using diamond shaped paths. These examples use linked and not matrix representation.

The GNU make file is used for building automation. It handles generation of executables, which also gives dependency tracking. The dependent executables will be generated in tandem. It can also recompile the updated files while ignoring the unchanged files, therefore reducing build time.

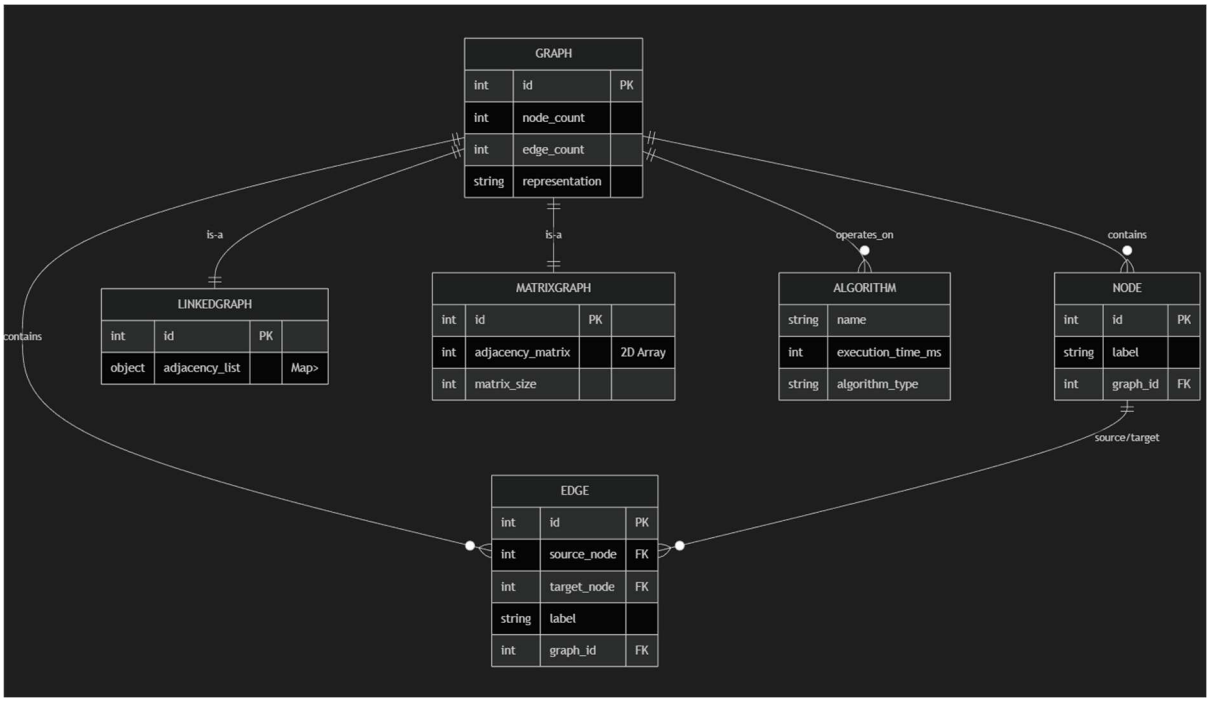
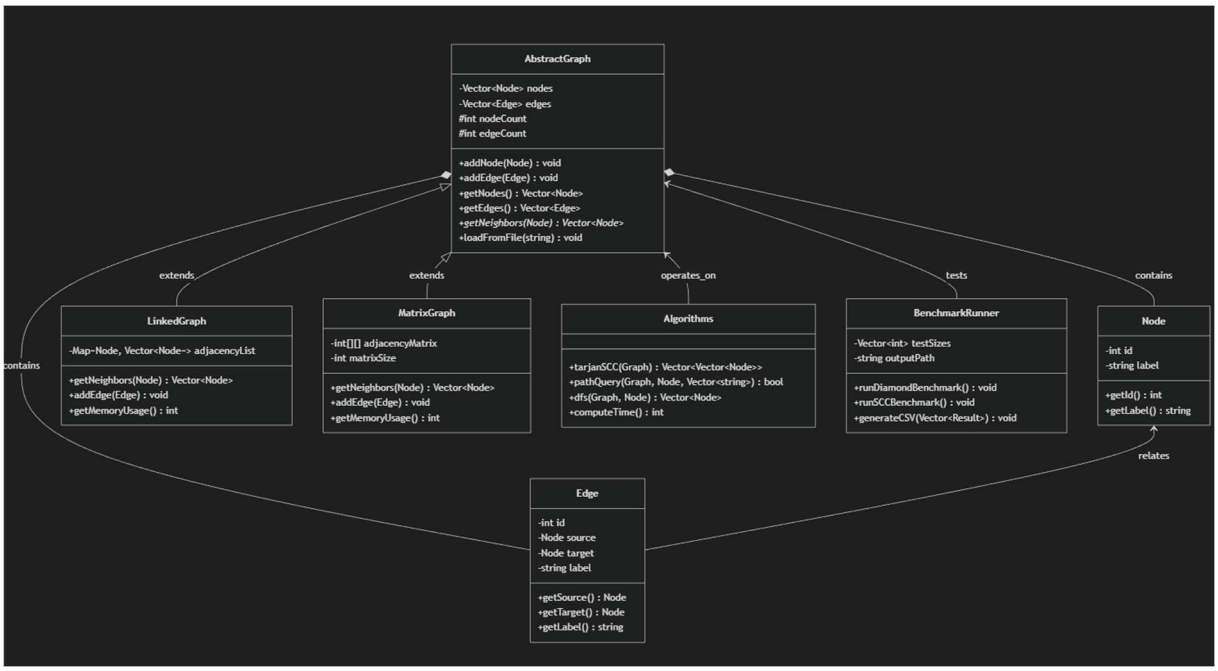
3.4 Scaling and runtime

The benchmarking is run by benchmark-runner.cpp, which measure algorithm performance. The graph size is hardcoded to n = 100, 500, 1000, 5000, 10000 nodes, and both representations are tested. The scenarios tested are diamond constant (m=5), diamond proportional (m=n/10) and SCC algorithm with random graphs.

When running the file, graph-benchmark.cpp and graph-benchmark-scc.cpp renamed generate-... will create test graphs to be used. Then the executed algorithms will be timed and stored in the CSV file, results/benchmark_results.csv. This process is due to the plotting of the data. We had issues with plotting using the internal tools for C++ plotting. To ease the data visualization a python file is used instead, plot_benchmarks.py. This file reads the csv filed and generates plots



which are outputted to the results folder. The issue is that the python file will not run automatically and needs to be called/run manually.

With constant length of query, there is no real observed difference between the two algorithms. With a higher constant you would probably see more of a difference as seen in the proportional plot. As earlier described, you can observe how the matrix representation is performing better for the dense graphs. The constant lookup time of O(1) helps for the small graphs, but as m and n increases the matrix representation increases the memory access needed and therefore has more difficulties. Any higher n could also result in the memory being exceeded as matrix representation needs O(n^2) memory allocated while linked only needs O(n). The SCC algorithm performs best with the matrix representation, this might be due to the lookup time being faster, but results could be different with higher n than tested. This is because in the matrix representation all cells are iterated through even the empty ones, while linked will only visit existing edges. This also explains why linked are performing better on sparse graphs.

3.5 ER diagram data + class